

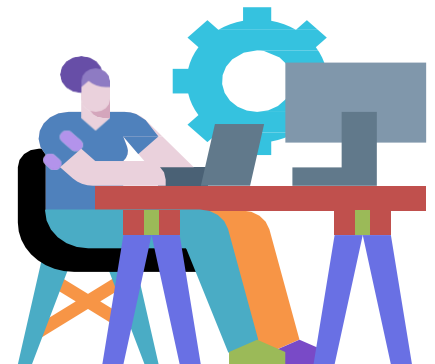
Lesson 1

Introduction to Python Programming Language



INTRODUCTION TO PYTHON

- Overview
- Coding Standard introduction
- Keywords
- Statement, Indentation and Comments
- Variables and constants



WHY PYTHON?

- ❑ **Object oriented language (Lesson 8):** Programming paradigm that relies on the concept of classes and objects. It is used to structure a software program into simple, reusable pieces of code.
- ❑ **Supports dynamic data type:** No need to declare a variable's type explicitly ; it can change at runtime.
- ❑ **Independent from platforms:** Can run on different operating systems and hardware platforms without modification
- ❑ **Simple and easy grammar:** Syntax is designed to be straightforward and readable, which makes it easier to learn and use.
- ❑ **High-level internal object data types:** Provides built-in support for complex data types.
- ❑ **Automatic memory management:** Automatically handles the allocation and deallocation of memory.
- ❑ **It's open source:** Source code is freely available for anyone to view, modify, and distribute

WHAT CAN PYTHON DO?

- ❑ Python can be used on a server to create web applications
- ❑ Python can be used alongside software to create workflows
- ❑ Python can connect to database systems
- ❑ It can also read and modify files
- ❑ Python can be used to handle big data and perform complex mathematics
- ❑ Python can be used for rapid prototyping, or for production-ready software development

Here is some of the areas you can use python for:

- ☐ Graphics
- ☐ System management (i.e., scripting)
- ☐ Graphic User Interface (GUI)
- ☐ **Cybersecurity**
- ☐ Data science
- ☐ Web applications
- ☐ Artificial Intelligence
- ☐ Numerical operations

- ☐ And so on...

TRY IT OUT

We have two variations of interactive Python Shells. These are

1. IDLE (GUI)
2. Python (command line)

In this training, we are going to be using Pycharm IDE as our development environment and we will be using python version 3.10.4.

An IDE (Integrated Development Environment) is used to edit and compile code. Pycharm is an integrated development environment (IDE) from jetbrains (formerly known as IntelliJ). It can be used to develop Python, web, and data science development.

Community is free and can be download from the link below

<https://www.jetbrains.com/pycharm/download/#section=windows>

FIRST PROGRAM

Type the following code in any text editor and save it as `hello_world.py`

```
print("Hello, world!")
```

Congratulations! You just wrote your first program in Python.

PYTHON CODING STANDARD

Coding standards are **collections of rules and guidelines that determine the programming style, procedures, and methods for a programming language.**

Benefits of coding standards

- Easy team integration
- Increased code quality efficiency and easy for maintaining
- Reduce code complexity
- Reduce development cost

Drawbacks of not having proper coding conventions for a team

Without predefined conditions that all team member should follow can result in the following:

- Reduced engineers motivation
- Increased development time
- Complex codebase structure

There isn't only one 'style'

- ❑ Most common: PEP8
 - ❑ Python Enhancement Proposal #8
 - ❑ Tools for checking the style adherence
 - ❑ conda install pep8
- ❑ Google Python Style Guide
 - ❑ *Python is the main scripting language used at Google. This style guide is a list of dos and don'ts for Python programs.*
 - ❑ Vim & Emacs 'plugins' for style adherence

PYTHON KEYWORDS

Keywords are the reserved words in Python.

We cannot use a keyword as a variable name, function name or any other identifier. They are used to define the syntax and structure of the Python language.

In Python, keywords are case sensitive.

There are 35 keywords in Python 3.8. This number can vary slightly over the course of time.

All the keywords except True, False and None are in lowercase and they must be written as they are. The next page will present the list of keywords.

PYTHON KEYWORDS(CONT.)

FALSE	class	from	or
None	continue	global	pass
TRUE	def	if	raise
and	del	import	return
as	elif	in	try
assert	else	is	while
async	except	lambda	with
await	finally	nonlocal	yield
break	for	not	



You can get a list of available keywords by using `help()`

`help("keywords")`

We can then use `help()` again by passing in the specific keyword that you need more information about. You can do this, for example, with the `pass` keyword:

`help("break")`

COMPLETE THE FOLLOWING QUESTION (5 MINS)

Use the help function to get more information regarding the following the following keywords

- Yield
- While
- And
- True

E.g `help("break")`

PYTHON STATEMENT, INDENTATION AND COMMENTS

Statements are instructions that a Python interpreter can execute. For example, `x = 10` is an assignment statement, and `for` statement, `while` statement, etc. are other kinds of statements.

Multi-line statement

This can be represented either by with a new line symbol `"\`" or explicit line continuation such as brackets `[ ]`, braces `{ }` and parentheses `( )` or using semicolons

Examples

```
X= 3+4+5+ \  
    6+8
```

```
X= (3+4+5+  
    6+8 )
```

```
X= 3;y=4; z=8
```

PYTHON STATEMENT, INDENTATION AND COMMENTS

Python uses indentation to define a block of code

A code block starts with indentation and ends with the first unindented line. The amount of indentation is up to you, but it must be consistent throughout that block.

```
if True:
    x = 7
    print('Hello')
```

Comments help describe what is going on inside a program, so that another person looking at the code doesn't have hard time. Comment can either be single or multiline.

```
#This is a comment
# Yes it is
print('Hello')
```

```
"""This is a comment
Yes it is """
print('Hello')
```

```
""" This is a comment
Yes it is """
print('Hello')
```

This can also be used as a Docstrings. A docstring is short for documentation string.

PYTHON VARIABLES

“Variables” are simply storage locations for data. You can place data into them and retrieve their contents as part of a python expression. Python gives the reference of the object (value) to the variable.

Declaring (Creating) Variables

```
variableName = value;
```

```
Example name = "John";
```

```
firstname = "john"  
print(firstname)
```

Note: Python is a type-inferred language, so you don't have to explicitly define the variable type. It automatically knows that **john** is a string and declares the **firstname** variable as a string.

Assigning multiple values to multiple variables

```
Firstname, lastname = "john","Bello"  
print(firstname)  
print(lastname)
```

Assigning same value to multiple variables

```
Firstname=lastname = "john"  
print(firstname)  
print(lastname)
```

Whitespace

- ❑ Multiple statements on the same line are discouraged

WRONG:if foo == 'blah': doBlahThing()

CORRECT:if foo == 'blah':
doBlahThing()

- ❑ No white space immediately before an open parenthesis

WRONG:course [1]

CORRECT:course(1)

- ❑ No white space inside parentheses, brackets or braces

- ❑ No more than one space around an operator

Block Comments:

- ❑ They are indented to the same level as the code they apply to
- ❑ Each line of a block comment starts with a # and a single space
- ❑ Paragraphs inside a block comment are separated by a line containing a single #
- ❑ Block comments are best surrounded by a blank line above and below them

Example:

```
# Compensate for border. This is done by incrementing x  
# by the same amount  
x += 1
```

Inline Comments:

- ❑ They should start with a # and a single space
- ❑ Should be separated by at least two spaces from the statement they apply to

Example:

```
x += 1# Compensate for border
```

PEP8 - DOCSTRINGS CODING STANDARD

- ❑ Write docstrings for all public modules, functions, classes, and methods (lesson 3 and 8)
- ❑ Docstrings are not necessary for non-public methods, but you should have a comment that describes what the method does
- ❑ This comment should appear after the "def" line
- ❑ Insert a blank line before and after all docstrings that document a class

One- line Docstrings:

- ❑ The opening and closing `"""` are on the same line
- ❑ There is no blank line either before or after the docstring
- ❑ Describes the function or method's effect as a command ("Do this", "Return that"), not as a description

Multi- line Docstrings:

- ❑ The `"""` that ends a multiline docstring should be on a line by itself
- ❑ Script: The docstring of a script should be usable as its "usage" message. It should document the script's function, the command line syntax, and the environment variables.
- ❑ Module: The docstring for a module should generally list the classes, exceptions and functions (and any other objects) that are exported by the module, with a one-line summary of each.

COMPLETE THE FOLLOWING QUESTION (10 MINS)

1. Write a docstring document with the sentence “Assigning same value to multiple variables”
2. Create two variables and assign the same value (Good day) at the same time to them
3. Add a comment “this prints the two assigned variables) above the print function.
4. Print your two variables

RULES & NAMING CONVENTION FOR VARIABLES AND CONSTANTS

- ❑ Create a name that makes sense
- ❑ If you want to create a variable name that has two words, use underscore to separate them
- ❑ Don't start a variable name with a digit
- ❑ Use capital letters possible to declare a constant
- ❑ Never use special symbols like !, @, #, \$, %, etc.

OTHER KEY NOTES

Type function

Examples

```
x = 5
```

```
y = "John"
```

```
print(type(x))
```

```
print(type(y))
```

explicitly Cast

You can explicitly Cast your datatype

```
x = str(3) # x will be '3'
```

```
y = int(3) # y will be 3
```

```
z = float(3) # z will be 3.0
```

NB:

1. String values can be either by single or double Quotes
2. Python is Case-sensitive.

Print Functions Examples

In the print() function, you output multiple variables, separated by a comma:

```
x = "Python"
```

```
y = "is"
```

```
z = "Greate"
```

```
print(x, y, z) #You can also use the comma
```

```
print(x + y + z) #You can also use the + operator to output multiple variables
```

```
x = 5
```

```
y = "Willis"
```

```
print(x + y) - You cant combine int and string together
```

The best way to output multiple variables in the print() function is to separate them with commas

```
x = 5
```

```
y = "John"
```

```
print(x, y)
```

LITERALS

Literal is a raw value or data that are stored in a variable or constant (ex. $\pi=3.14$). In Python there are various types of literals, they are:

- Numeric Literals
- Boolean literals
- String literals
- Special literals
- Literal Collections

NUMERICAL LITERALS

Numeric Literals are immutable (unchangeable). Numeric literals can belong to 3 different numerical types: Integer, Float, and Complex.

```
a = 0b1010 #Binary Literals
b = 200 #Decimal Literal
c = 0o310 #Octal Literal
d = 0x12c #Hexadecimal Literal
```

```
# Integer Literal
```

```
int_1 = 36
```

```
int_2 = -12
```

```
#Float Literal
```

```
float_1 = 10.34
```

```
float_2 = 3.5e2
```

```
#Complex Literal
```

```
x = 3.14j
```

```
print(a, b, c, d)
```

```
print(float_1, float_2)
```

```
print(x, x.imag, x.real)
```

When we print the variables,
all the literals are converted
into decimal values.

Output:

```
10 200 200 300
10.34 350.0
3.14j 3.14 0.0
```

BOOLEAN LITERALS

This can only have one of two values, such as : True or false

```
x = True  
x = False  
  
print("X is", x)  
print("Y is", y)
```

Output:
X is true
Y is False

STRING LITERALS

A string literal is a sequence of characters surrounded by quotes. We can use both single, double, or triple quotes for a string.

```
strings = "This is Python"
char = "C"
multiline_string = """This is a multiline string
with more than one line code."""
unicode = u"\ u00dcnic\ u00f6de"
raw_str = r"raw \n string"

print(strings)
print(char)
print(multiline_string)
print(unicode)
print(raw_str)
```

String literal

character literal

Multi-line literal

Output:

```
This is Python
C
This is a multiline string with more than one line code.
Unicode
raw \n string
```

SPECIAL LITERALS

Python contains one special literal i.e. None. We use it to specify that the field has not been created.

```
event_status = "Available"  
default_status = None
```

```
def tour(x):  
    if x == event_status:  
        print(event_status)  
    else:  
        print(default_status)
```

```
tour(event_status)  
tour(default_status)
```

Output:
Available
None

Operation Methods in string

- upper()
- lower()
- capitalize()
- count(s)
- find(s)
- rfind(s)
- index(s)
- strip(), lstrip(), rstrip()
- replace(a, b)
- expandtabs()
- split()
- join()
- center(), ljust(), rjust()
- Encode()
- Format()

COMPLETE THE FOLLOWING QUESTION

1. Create a variable and assign “What a wonderful day” to it.
2. Using the following operation method, print the value of to the console.
 1. Capitalize the sentence
 2. Find the index of day
 3. Replace day with evening

PYTHON ESCAPE SEQUENCE AND ESCAPE CHARACTER

Escape sequence

- An escape sequence is a special character used in the form of backslash(\) followed by a character that is required.
- These characters are used to represent whitespace.
- Whitespace gives characters like space, tab, formfeed, vertical tab.

	Meaning
\'	This represents a single quote
\n	This represents a newline
\r	This represents a carriage return

	Meaning
\t	This represents a tab
\b	This represents a backspace
\f	This represents a formfeed

Escape Character

To insert characters that are illegal in a string, use an escape character.

An escape character is a backslash \ followed by the character you want to insert.

```
sentence = "Today is a new day and it is "Thursday""
```

PYTHON STRING FORMAT()

Python Syntax	Description
:<	Left aligns the result (within the available space)
:>	Right aligns the result (within the available space)
:^	Center aligns the result (within the available space)
:=	Places the sign to the left most position
:+	Use a plus sign to indicate if the result is positive or negative
:	Use a space to insert an extra space before positive numbers (and a minus sign before negative numbers)
:,	Use a comma as a thousand separator
_	Use a underscore as a thousand separator
:b	Binary format
:c	Converts the value into the corresponding unicode character
:d	Decimal format
:e	Scientific format, with a lower case e
:f	Fix point number format
:g	General format
:o	Octal format
:x	Hex format, lower case
:n	Number format
:%	Percentage format

COMPLETE THE FOLLOWING QUESTION

1. Write a python script that prints the counts the total of "o" in the below string

"Python scripting is good. Python is used across a wide variety of industries."

2. Write a python script that removes the all the "-" before and after python training

```
python_sentence = "-----python training-----  
-----"
```


COMPLETE THE FOLLOWING QUESTION

Create the below variables that store the following information

quality = 10000

total = 1507500

price = 150.75

Using the values above, format your print function to format and display to the below sense

It costed me \$1,507,500 to by 10,000 quality of office supplies in "Ottawa Campus". Individual Price of the item was 150.75.

Please make sure you use the variable created in your print formation

LITERALS CONNECTIONS

There are four different literal collections.

List literals

Dictionary literals

Tuple literals

Set literals

```
fruits = ["apple", "mango", "orange"]  
numbers = (1, 2, 3)  
alphabets = {'a': 'apple', 'b': 'ball', 'c': 'cat'}  
  
vowels = {'a', 'e', 'i', 'o', 'u'}
```

```
print(fruits)  
print(numbers)  
print(alphabets)  
print(vowels)
```

List

Tuple

Dictionary

set

Output:

```
['apple', 'mango', 'orange']  
(1, 2, 3)  
{'a': 'apple', 'b': 'ball', 'c': 'cat'}  
{'e', 'a', 'o', 'i', 'u'}
```

DATA TYPES

- ❑ **Data types define the kind of value that a variable can hold**
- ❑ Every value in Python has a datatype
- ❑ Everything is an object in Python programming
- ❑ Data types are actually classes and variables are instance (object) of these classes

Python Numbers

Python Tuple

Python List

Python Strings

Python Dictionary

Python Set

Python Numbers

Integers, floating point numbers and complex numbers fall under Python numbers category. They are defined as int, float and complex classes in Python.

DATA TYPES (CONT.)

Python Numbers

Integers, floating point numbers and complex numbers fall under Python numbers category. They are defined as int, float and complex classes in Python.

We can use the **type()** function to know which class a variable or a value belongs to. Similarly, the **isinstance()** function is used to check if an object belongs to a particular class.

Integers can be of any length; it is only limited by the memory available.

Floating-point number is accurate up to 15 decimal places. Integer and floating points are separated by decimal points.

Complex numbers are written in the form, $a + bj$, where a is the real part and b is the imaginary part.

DATA TYPES (CONT.)

Python List

List is an ordered sequence of items. It is one of the most used datatype in Python and is very flexible. All the items in a list do not need to be of the same type.

```
fruits = ["apple", "mango", "orange", "banana", "grape"]  
  
print(" fruits[1] = ", fruits[1])  
print(" fruits[0:3] = ", fruits[0:3])  
print(" fruits[2:] = ", fruits[2:])
```

Output:

What will be the output?

Operation method in List

- | | |
|----------|-----------|
| ▪ append | ▪ reverse |
| ▪ insert | ▪ pop |
| ▪ index | ▪ Extend |
| ▪ count | ▪ Del |
| ▪ sort | ▪ Clear |
| ▪ remove | ▪ Copy |

The index starts from 0 in Python.

To determine how many items a list has, use the **len()** function

COMPLETE THE FOLLOWING QUESTION

1. Create a list of cars(Honda, Toyota, Mercedes, Ferrari, Nissan, Hyundai)
2. Print the list
3. Append a new car “Ford” to the list and print the list
4. Print Mercedes only on the screen
5. Find and print the index of Ferrari
6. Remove Honda from the list
7. Print the new list

COMPLETE THE FOLLOWING QUESTION

1. Create a list of cars(Honda, Toyota, Mercedes, Ferrari, Nissan, Hyundai)
2. Print the list
3. Change the last item to “Kia”
Tips: Reference to your index value should be dynamic. This will ensure that the value does not change even if a new list member is added, or an existing list member of the list is deleted.
4. Print the list
5. Add a new member “Chevrolet” to index 2 of the list
Expected result (Honda, Toyota, Chevrolet, Mercedes, Ferrari, Nissan, Kia)
6. Print the list
7. Reverse the list in descending order
8. Print the list
9. Empty all the members of the list
10. Print the list

ITERATING THROUGH A LIST

Loop Through a List

You can loop through the list items by using a for loop

Syntax

```
for localVariable in ListvariableName:  
    print(localVariable)
```

Loop Through the Index Numbers

We can also loop through index

Use the range() and len() functions to create a suitable iterable.

Syntax

```
for indexCounter in range(len(ListvariableName)):  
    print(ListvariableName[indexCounter])
```


ITERATING THROUGH A LIST (CONT.)

Looping Using List Comprehension

List Comprehension offers the shortest syntax for looping through lists.

Syntax

```
[print(localVariable) for localVariable in ListvariableName]
```

COMPLETE THE FOLLOWING QUESTION

1. Create a list of cars(Honda, Toyota, Mercedes, Ferrari, Nissan, Hyundai)
2. Print the list member on each line of your output using the for loop via the index
3. Print the list member on each line of your output using the for loop
4. Print the list member on each line of your output using the for list Comprehension

NB: List comprehension offers a shorter syntax when you want to create a new list based on the values of an existing list.

DATA TYPES (CONT.)

Python Tuple

Tuple is an ordered sequence of items same as a list. The only difference is that tuples are immutable. Tuples once created cannot be modified.

Tuples are used to write-protect data and are usually faster than lists as they cannot change dynamically.

```
tuplevariable = (15,'Willis', 4+3j)
print(" tuplevariable[1] = ",tuplevariable[1])
```

Output:

What will be the output?

Negative Indexing

Negative indexing means start from the end.

-1 refers to the last item, -2 refers to the second last item etc.

Range of Indexes

You can specify a range of indexes by specifying where to start and where to end the range

```
tupleVariableName[2:6]
```

Note: The search will start at index 2 (included) and end at index 6 (not included).

Updating a tuple

Tuple are not unchangeable once they are created. However, the work around is to convert your tuple to list and perform any information and convert back to a tuple.

PYTHON TUPLE

Unpacking a Tuple

This is the processing of assigning a variable to all members of the tuple.

Note: The number of variables must match the number of values in the tuple, if not, you must use an asterisk to collect the remaining values as a list.

Join Two Tuples

To join two or more tuples you can use the `+` operator or `+=` operator

Multiply Tuples

If you want to multiply the content of a tuple a given number of times, you can use the `*` operator

Delete Tuples The `del` keyword can delete the tuple completely

Tuple Methods

Count -> Returns the number of times a specified value occurs in a tuple

Index -> Searches the tuple for a specified value and returns the position of where it was found

COMPLETE THE FOLLOWING QUESTION (15 MINS)

1. Create a tuple of fruits(banana, Apple, Mango, Grape, Orange) and call it fruit_tuple_one
2. Print the tuple
3. Print Mango only on the screen
4. Unpack fruit_tuple_one and print all the variables
5. Count the number of times “Grape” occurred in the tuple
6. Create another tuple of (Kiwi, Lemon, Lime) and call it fruit_tuple_two
7. Add the two list members and print
8. Find the index of "Lemon" in the combined tuple

ITERATING THROUGH A TUPLE

Loop Through a tuple

You can loop through the tuple items by using a for loop

Syntax

```
for localVariable in tuplevariableName:  
    print(localVariable)
```

Loop Through the Index Numbers

We can also loop through index

Use the range() and len() functions to create a suitable iterable.

Syntax

```
for indexCounter in range(len(tuplevariableName)):  
    print(tuplevariableName[indexCounter])
```

DATA TYPES (CONT.)

Python Dictionary

Dictionary in Python is an unordered collection of data values, used to store data values like a map, which, unlike other Data Types that hold only a single value as an element, Dictionary holds key:value pair. Key-value is provided in the dictionary to make it more optimized.

```
# Creating a Dictionary
# with Integer Keys
Dict = { 1: 'Geeks', 2: 'For', 3: 'Geeks'}
print("\nDictionary with the use of Integer Keys: ")
print(Dict)
print(Dict.keys())

# Creating a Dictionary
# with Mixed keys
Dict = { 'Name': 'Geeks', 1: [1, 2, 3, 4]}
print("\nDictionary with the use of Mixed Keys: ")
print(Dict)

# Creating a Dictionary
# with Integer Keys
Dict = { 1: 'Geeks', 2: 'For', 3: 'Geeks'}
print("\nDictionary with the use of Integer Keys: ")
print(Dict)

# Creating a Dictionary
# with Mixed keys
Dict = { 'Name': 'Geeks', 1: [1, 2, 3, 4]}
print("\nDictionary with the use of Mixed Keys: ")
print(Dict)
```

Output:

What will be the output?

DATA TYPES (CONT.)

Operation method in Dictionary

Methods	Description
<code>clear()</code>	Removes all the elements from the dictionary
<code>copy()</code>	Returns a copy of the dictionary
<code>fromkeys()</code>	Returns a dictionary with the specified keys and value
<code>get()</code>	Returns the value of the specified key
<code>items()</code>	Returns a list containing a tuple for each key value pair
<code>keys()</code>	Returns a list containing the dictionary's keys
<code>pop()</code>	Removes the element with the specified key
<code>popitem()</code>	Removes the last inserted key-value pair
<code>setdefault()</code>	Returns the value of the specified key. If the key does not exist: insert the key, with the specified value
<code>update()</code>	Updates the dictionary with the specified key-value pairs
<code>values()</code>	Returns a list of all the values in the dictionary

COMPLETE THE FOLLOWING QUESTION

- Create a Dictionary for Ford car using the below properties
 "brand": "Ford",
 "model": "Mustang",
 "year": 2022
- Print only the keys (Ford, Mustang, 2022)
- Print tuple for each key value pair all item

DATA TYPES (CONT.)

Python Strings

Python String is sequence of Unicode characters. We can use single quotes or double quotes to represent strings. Multi-line strings can be denoted using triple quotes, `'''` or `"""`. Strings are immutable.

Python Set

Python Set is an unordered collection of unique items. Set is defined by values separated by comma inside braces `{ }`. Items in a set are not ordered.

```
school = 'Willis College!'
print(" school[3] = ", school[4])

print(" school[1:6] = ", school[1:6])

# Generates error
# Strings are immutable in Python
school[5] = 'd'

a = {5,2,3,1,4}

# printing set variable
print("a = ", a)

# data type of variable a
print(type(a))

# Generates error : slicing operator [] does not work
print("a = ", a[2])
```

Since, sets are unordered collection, indexing has no meaning. Hence, the slicing operator `[]` does not work

Output:

```
school[4] = i
school[1:6] = illis
a = {1, 2, 3, 4, 5}
<class 'set'>
```

DATA TYPES (CONT.)

Conversion between data types

We can convert between different data types by using different type conversion functions like `int()`, `float()`, `str()`, etc.

```
print(float(5))
```

Convert to float

```
print(int(-
```

Convert to integer

```
print(str(25))
```

Convert to string

```
print(set([1,2,3]))
```

Convert to set

```
print(tuple({5,6,7}))
```

Convert to tuple

```
print(list('hello'))
```

Convert to list

Output:

5.0

-10

25

{1, 2, 3}

(5, 6, 7)

['h', 'e', 'l', 'l', 'o']

PYTHON SET

Python Set

Duplicates Not Allowed

Sets cannot have two items with the same value.

Add items to Set

Once a set is created, you cannot change its items, but you can add new items.

Remove Item

To remove an item in a set, use the `remove()`, or the `discard()` method. Remove will raise an error if item does not exist while discard will not.

Add Any Iterable

The object in the `update()` method does not have to be a set, it can be any iterable object (tuples, lists, dictionaries etc.).

Joining a Set

You can use the `union()` method that returns a new set containing all items from both sets, or the `update()` method that inserts all the items from one set into another

NB: Both `union()` and `update()` will exclude any duplicate items

PYTHON SET METHODS

Method	Description
<code>add()</code>	Adds an element to the set
<code>clear()</code>	Removes all the elements from the set
<code>copy()</code>	Returns a copy of the set
<code>difference()</code>	Returns a set containing the difference between two or more sets
<code>difference_update()</code>	Removes the items in this set that are also included in another, specified set
<code>discard()</code>	Remove the specified item
<code>intersection()</code>	Returns a set, that is the intersection of two other sets
<code>intersection_update()</code>	Removes the items in this set that are not present in other, specified set(s)
<code>isdisjoint()</code>	Returns whether two sets have a intersection or not
<code>issubset()</code>	Returns whether another set contains this set or not
<code>issuperset()</code>	Returns whether this set contains another set or not
<code>pop()</code>	Removes an element from the set
<code>remove()</code>	Removes the specified element
<code>symmetric_difference()</code>	Returns a set with the symmetric differences of two sets
<code>symmetric_difference_update()</code>	inserts the symmetric differences from this set and another
<code>union()</code>	Return a set containing the union of sets
<code>update()</code>	Update the set with the union of this set and others

COMPLETE THE FOLLOWING QUESTION

- Create two sets with the below details
 - Convert List_rating_one list to set with these details [1, 2, 3, 4, 5] and call it set_rating_one
 - Create the second set with these details {5,6,7,8,9,10} and call it set_rating_two
- Add Number 11 and 12 to set_rating_two
- Print the two sets.
- Returns a set that contains only members that exist in set_rating_one but not in set_rating_two and print your result
Tips: output should be {1, 2, 3, 4}
- Copy the set members of set_rating_two to another set and call it set_rating_three
- Remove “8” from set_rating_three
- Try removing “20” from set_rating_three without your script returning an error.
- Print the set_rating_three after removals

TYPE CONVERSION

Type conversion is converting one type of data to another type. It is also known as Type Casting. In Python type casting has two forms:

Implicit type conversion

Explicit type conversion

```
num_int = 123
num_flo = 1.23

num_new = num_int + num_flo

print("datatype of num_int:",type(num_int))
print("datatype of num_flo:",type(num_flo))

print("Value of num_new:",num_new)
print("datatype of
num_new:",type(num_new))
```

Implicit type conversion – These conversions are performed by the interpreter in a type-safe manner. Implicit casting is done automatically when passing a smaller size type to a larger size type:

Output:

```
datatype of num_int: <class 'int'>
datatype of num_flo: <class 'float'>
Value of num_new: 124.23
datatype of num_new: <class 'float'>
```


TYPE CONVERSION (CONT.)

```
num_int = 123
num_str = "456"

print("Data type of num_int:",type(num_int))
print("Data type of num_str before Type Casting:",type(num_str))

num_str = int(num_str)
print("Data type of num_str after Type Casting:",type(num_str))

num_sum = num_int + num_str

print("Sum of num_int and num_str:",num_sum)
print("Data type of the sum:",type(num_sum))
```

Explicit type conversion –
Explicit conversions require a cast operator

Output:

```
Data type of num_int: <class 'int'>
Data type of num_str before Type Casting: <class 'str'>
Data type of num_str after Type Casting: <class 'int'>

Sum of num_int and num_str: 579
Data type of the sum: <class 'int'>
```

KEY POINTS TO REMEMBER

- ❑ Type Conversion is the conversion of object from one data type to another data type
- ❑ Implicit Type Conversion is automatically performed by the Python interpreter
- ❑ Python avoids the loss of data in Implicit Type Conversion
- ❑ Explicit Type Conversion is also called Type Casting, the data types of objects are converted using predefined functions by the user
- ❑ In Type Casting, loss of data may occur as we enforce the object to a specific data type

PYTHON INPUT, OUTPUT AND IMPORT

Input

We use the `input([prompt])` function to collect input from the user. where `prompt` is the string we wish to display on the screen. It is optional.

```
Age = input('Enter a age: ')
```

Python Import

When our program grows bigger, it is a good idea to break it into different modules. A module is a file containing Python definitions and statements. Python modules have a filename and end with the extension “.py”.

The `import` keyword be imported to another module or the interactive interpreter in Python to another module

```
import math  
print(math.pi)
```

```
from math import pi  
pi
```

We can also import some specific attributes and functions only, using the `from` keyword

`sys.path` can be used get the list of directory locations when importing a module

PYTHON INPUT, OUTPUT AND IMPORT

Output

We use the `print()` function to output data to the standard output device (screen).

```
print(1, 2, 3, 4)
print(1, 2, 3, 4, sep='#', end='&')
```

Print parameters:

sep: Specify how to separate the objects, if there is more than one. Default is ' ' (space character)

end : Specify what to print at the end. Default is '\n'

Output formatting

Sometimes we would like to format our output to make it look attractive. This can be done by using the `str.format()` method. This method is visible to any string object.

```
x = 6; y = 20
>>>print('The value of x is {0} and y is {1}'.format(x,y))
```

This can be optional

PYTHON OPERATORS

Operators are used to perform operations on variables and values.

Arithmetic Operators

Assignment operators

special Operators

Comparison Operators

Logical Operators

Arithmetic Operators: Arithmetic operators are used to perform common mathematical operations

Operator	Meaning	Example
+	Add two operands or unary plus	$x + y + 2$
-	Subtract right operand from the left or unary minus	$x - y - 2$
*	Multiply two operands	$x * y$
/	Divide left operand by the right one (always results into float)	x / y
%	Modulus - remainder of the division of left operand by the right	$x \% y$ (remainder of x/y)
//	Floor division - division that results into whole number adjusted to the left in the number line	$x // y$
**	Exponent - left operand raised to the power of right	$x ** y$ (x to the power y)

PYTHON OPERATORS (CONT.)

Comparison Operators

```
x = 15  
y = 4
```

```
print('x + y =',x+y)  
print('x - y =',x-y)  
print('x * y =',x*y)  
  
print('x > y is',x>y)  
print('x < y is',x<y)
```

Operator	Name	Example
==	Equal to	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

Output:
x + y = 19
x - y = 11
x * y = 60
x > y is True
x < y is False

PYTHON OPERATORS (CONT.)

Assignment operators: Assignment operators are used to assign values to variables

Operator	Example	Same As
=	x = 19	x = 19
+=	x += 5	x = x + 5
-=	x -= 7	x = x - 7
*=	x *= 7	x = x * 7
/=	x /= 8	x = x / 8
%=	x %= 2	x = x % 2
&=	x &= 3	x = x & 3
=	x = 4	x = x 4
^=	x ^= 4	x = x ^ 6
>>=	x >>= 6	x = x >> 6
<<=	x <<= 6	x = x << 6

PYTHON OPERATORS (CONT.)

Logical operators are the and, or, not operators

Operator	Meaning	Example
and	True if both the operands are true	x and y
or	True if either of the operands is true	x or y
not	True if operand is false (complements the operand)	not x

PYTHON OPERATORS (CONT.)

Special operators

Python language offers some special types of operators like the identity operator or the membership operator

Operator	Meaning
is	True if the operands are identical
is not	True if the operands are not identical
in	True if value/variable is found in the sequence
not in	True if value/variable is not found in the sequence

```
x = 5  
y = 5  
z = "Hello"
```

```
# Output: True  
print(x is y)
```

```
# Output: True  
print(x is not z)
```

```
# Output: True  
print('H' in z)
```

COMPLETE THE FOLLOWING QUESTION

1. Create two variables and assign 5 and 7 respectively
2. Add and store the sum of the two numbers
3. Format your output to the below
“The sum of 5 and 7 is 12”
4. Convert to code to read two input from the user and ensure your variables are dynamically mapped to the output
“The sum of ***firstinput*** and ***secondinput*** is ***sum***”